Linguaggi e Metodologie di Programmazione - Primo Emicorso

> **Corso:** LMP - Programmazione Orientata agli Oggetti (Java)
> **Università:** Tor Vergata
> **Libro di riferimento:** *Thinking in Java* (4th Edition) - Bruce Eckel
> **Tag:** #Java #OOP #Programmazione #Università #LMP

Indice

1. Introduzione al Paradigma Object-Oriented
2. Classi e Oggetti in Java
3. Operatori in Java
4. Controllo del Flusso
5. Inizializzazione e Cleanup
6. Controllo dell'Accesso
7. Riuso di Classi (Composizione ed Ereditarietà)
8. Polimorfismo
9. Interfacce
10. Classi Interne
11. Strutture Dati: Collections Framework
12. Gestione degli Errori: Eccezioni
13. Meta-Informazioni: Reflection
14. Generics
15. Input/Output (I/O)
16. Tipi Enumerati
17. Annotazioni (Meta-programmazione)
18. Programmazione Concorrente
19. Maven: Gestione Progetti e Dipendenze

---

1. Introduzione al Paradigma Object-Oriented

1.1 Cos'è la Programmazione Orientata agli Oggetti

La **programmazione orientata agli oggetti (OOP)** è un paradigma di programmazione che organizza il software intorno a **oggetti** piuttosto che funzioni o logica. Un oggetto è un'entità che combina **dati** (attributi/stato) e **comportamenti** (metodi) in un'unica unità.

I quattro pilastri fondamentali dell'OOP sono:

Pilastro	Descrizione	Analogia Reale
Incapsulamento	Nasconde i dettagli interni, espone solo l'interfaccia	Telecomando TV: premi il pulsante, non vedi i circuiti
Ereditarietà	Una classe eredita proprietà da un'altra	Bambino eredita tratti dai genitori
Polimorfismo	Stessa interfaccia, comportamenti diversi	Pedale dell'acceleratore funziona su ogni auto
Astrazione	Mostra solo le caratteristiche essenziali	Mappa stradale: mostra le strade, non ogni edificio

1.2 Java come Linguaggio "Puramente" ad Oggetti

Java è considerato **quasi** puramente orientato agli oggetti perché:

- **Tutto è un oggetto** (quasi): ogni classe estende implicitamente `java.lang.Object`
- I **tipi primitivi** (`int` , `double` , `boolean` , ecc.) non sono oggetti (ma hanno wrapper class)
- Il codice risiede sempre all'interno di classi
- Non esistono funzioni globali (solo metodi statici di classe)

```
// In Java, anche il main è dentro una classe
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, OOP World!");
    }
}
```

1.3 Evoluzione dei Linguaggi di Programmazione

```
Linguaggio Macchina (0/1)
    ↓
Assembly (mnemonici)
    ↓
Linguaggi Procedurali (C, Pascal) - focalizzati sulle funzioni
    ↓
Linguaggi Object-Oriented (Java, C++, C#, Python) - focalizzati sui dati
```

Vantaggi dell'OOP rispetto alla programmazione procedurale:

Aspetto	Procedurale	OOP
Organizzazione	Funzioni e dati separati	Dati e funzioni uniti in oggetti
Riutilizzo	Copia-incolla del codice	Ereditarietà e composizione
Manutenibilità	Difficile con codice grande	Modulare e scalabile
Sicurezza dati	Variabili globali accessibili	Incapsulamento con accesso controllato

---

2. Classi e Oggetti in Java

2.1 Classe: il "Progetto" dell'Oggetto

Una **classe** è un modello/template che definisce come saranno gli oggetti. È come una ricetta di cucina: descrive gli ingredienti (attributi) e i passaggi (metodi), ma non è il piatto finito.

```
// Dichiarazione di una classe
public class Automobile {
    // Attributi (stato dell'oggetto)
    private String marca;
    private String modello;
    private int velocita;
    private boolean motoreAcceso;

    // Costruttore: metodo speciale per creare l'oggetto
    public Automobile(String marca, String modello) {
        this.marca = marca;
        this.modello = modello;
        this.velocita = 0;
        this.motoreAcceso = false;
```

```
    }

    // Metodi (comportamenti)
    public void accendiMotore() {
        this.motoreAcceso = true;
        System.out.println(marca + " " + modello + ": Motore acceso!");
    }

    public void accelera(int incremento) {
        if (motoreAcceso) {
            velocita += incremento;
            System.out.println("Velocità: " + velocita + " km/h");
        } else {
            System.out.println("Errore: accendi prima il motore!");
        }
    }

    // Getter e Setter (incapsulamento)
    public String getMarca() {
        return marca;
    }

    public int getVelocita() {
        return velocita;
    }
}
```

2.2 Oggetto: l'Istanza della Classe

Un **oggetto** è un'istanza concreta di una classe, creato in memoria tramite il costruttore con la parola chiave `new`.

```
public class TestAutomobile {
    public static void main(String[] args) {
        // Creazione di oggetti (istanze)
        Automobile auto1 = new Automobile("Fiat", "Panda");
        Automobile auto2 = new Automobile("BMW", "X3");

        // Utilizzo degli oggetti
        auto1.accendiMotore();  // Fiat Panda: Motore acceso!
        auto1.accelera(50);     // Velocità: 50 km/h

        auto2.accendiMotore();  // BMW X3: Motore acceso!
        auto2.accelera(100);    // Velocità: 100 km/h

        // auto1 e auto2 sono oggetti INDIPENDENTI in memoria
        System.out.println(auto1.getVelocita()); // 50
        System.out.println(auto2.getVelocita()); // 100
    }
}
```

2.3 Diagramma di una Classe in UML

```
┌──────────────────────┐
│     Automobile       │
├──────────────────────┤
│ - marca: String      │
│ - modello: String    │
│ - velocita: int      │
│ - motoreAcceso: boolean
├──────────────────────┤
```

```
| + accendiMotore()   |
| + accelera(int)     |
| + getMarca(): String|
| + getVelocita(): int|
└─────────────────────┘

Legenda UML:
- = private
+ = public
# = protected
```

2.4 Il Riferimento `this`

La parola chiave `this` si riferisce all'**istanza corrente** dell'oggetto:

```java
public class Studente {
    private String nome;  // campo della classe

    public Studente(String nome) {  // parametro del costruttore
        this.nome = nome;  // this.nome = campo, nome = parametro
    }

    public void setNome(String nome) {
        this.nome = nome;  // distingue campo da parametro
    }

    public Studente getThis() {
        return this;  // restituisce l'oggetto corrente
    }
}
```

---

3. Operatori in Java

3.1 Operatori Aritmetici

Operatore	Descrizione	Esempio	Risultato
`+`	Addizione	`5 + 3`	`8`
`-`	Sottrazione	`5 - 3`	`2`
`*`	Moltiplicazione	`5 * 3`	`15`
`/`	Divisione	`5 / 3`	`1` (intera)
`%`	Modulo (resto)	`5 % 3`	`2`

```java
int a = 17;
int b = 5;

System.out.println("Somma: " + (a + b));        // 22
System.out.println("Differenza: " + (a - b));   // 12
System.out.println("Prodotto: " + (a * b));     // 85
System.out.println("Quoziente: " + (a / b));    // 3 (divisione intera!)
System.out.println("Resto: " + (a % b));        // 2

// ATTENZIONE: divisione tra interi tronca il decimale!
double risultato = 17 / 5;   // 3.0, NON 3.4!
double corretto = 17.0 / 5;  // 3.4 ✓
```

3.2 Operatori di Confronto

Operatore	Descrizione	Esempio
`==`	Uguale a	`a == b`
`!=`	Diverso da	`a != b`
`<`	Minore	`a < b`
`>`	Maggiore	`a > b`
`<=`	Minore o uguale	`a <= b`
`>=`	Maggiore o uguale	`a >= b`

3.3 Operatori Logici

Operatore	Descrizione	Esempio
`&&`	AND logico (entrambi veri)	`(a > 0) && (b > 0)`
`||`	OR logico (almeno uno vero)	`(a > 0) || (b > 0)`
`!`	NOT logico (negazione)	`!(a > 0)`

```
int eta = 25;
boolean haPatente = true;

// Può noleggiare un'auto?
boolean puoNoleggiare = (eta >= 18) && haPatente;
// true: ha più di 18 anni E ha la patente

// Sconto per studenti o anziani
boolean haSconto = (eta <= 25) || (eta >= 65);
// true: ha meno di 25 anni O più di 65 anni
```

3.4 Operatori di Assegnazione

```
int x = 10;

x += 5;   // x = x + 5;  → x = 15
x -= 3;   // x = x - 3;  → x = 12
x *= 2;   // x = x * 2;  → x = 24
x /= 4;   // x = x / 4;  → x = 6
x %= 4;   // x = x % 4;  → x = 2

// Incremento/Decremento
int y = 5;
int pre = ++y;   // pre-incremento: y diventa 6, pre = 6
int post = y++;  // post-incremento: post = 6, poi y diventa 7
```

3.5 Precedenza degli Operatori (dalla più alta alla più bassa)

1. `()` (parentesi)
2. `++`, `--` (unari)
3. `*`, `/`, `%`
4. `+`, `-`
5. `<`, `>`, `<=`, `>=`, `instanceof`
6. `==`, `!=`

7. `&&`
8. `||`
9. `=`, `+=`, `-=` (assegnazione)

---

4. Controllo del Flusso

4.1 if-else if-else

```
public class ControlloVoto {
    public static void main(String[] args) {
        int voto = 27;

        if (voto >= 30) {
            System.out.println("Perfetto! 30 e lode!");
        } else if (voto >= 27) {
            System.out.println("Ottimo!");
        } else if (voto >= 24) {
            System.out.println("Buono!");
        } else if (voto >= 18) {
            System.out.println("Sufficiente!");
        } else {
            System.out.println("Insufficiente!");
        }
    }
}
```

4.2 switch

Il `switch` confronta una variabile con valori costanti:

```
public class GiorniSettimana {
    public static void main(String[] args) {
        int giorno = 3;
        String nomeGiorno;

        switch (giorno) {
            case 1:
                nomeGiorno = "Lunedì";
                break;  // IMPORTANTE: esce dallo switch
            case 2:
                nomeGiorno = "Martedì";
                break;
            case 3:
                nomeGiorno = "Mercoledì";
                break;
            case 4:
                nomeGiorno = "Giovedì";
                break;
            case 5:
                nomeGiorno = "Venerdì";
                break;
            case 6:
            case 7:  // casi multipli con lo stesso risultato
                nomeGiorno = "Weekend!";
                break;
            default:
                nomeGiorno = "Giorno non valido";
        }
```

```
        System.out.println(nomeGiorno); // Mercoledì
    }
}
```

ATTENZIONE: Senza `break`, l'esecuzione "casca" nel case successivo (*fall-through*)!

4.3 Cicli: for, while, do-while

```
// FOR: quando sai quante iterazioni fare
for (int i = 0; i < 5; i++) {
    System.out.println("Iterazione " + i);
}

// Enhanced for (for-each): per scorrere collezioni
String[] nomi = {"Alice", "Bob", "Charlie"};
for (String nome : nomi) {
    System.out.println(nome);
}

// WHILE: quando la condizione è controllata prima
int contatore = 0;
while (contatore < 5) {
    System.out.println("Contatore: " + contatore);
    contatore++;
}

// DO-WHILE: quando la condizione è controllata dopo (almeno un'esecuzione)
int numero;
do {
    System.out.print("Inserisci un numero positivo: ");
    numero = scanner.nextInt();
} while (numero <= 0);
```

4.4 break e continue

```
// break: esce dal ciclo
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break;  // esce quando i == 5
    }
    System.out.println(i);  // stampa 0,1,2,3,4
}

// continue: salta all'iterazione successiva
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue;  // salta i numeri pari
    }
    System.out.println(i);  // stampa 1,3,5,7,9
}
```

---

5. Inizializzazione e Cleanup

5.1 Costruttori

Un **costruttore** è un metodo speciale chiamato automaticamente quando si crea un oggetto con `new`. Ha lo stesso nome della classe e **non ha tipo di ritorno**.

```
public class Persona {
    private String nome;
    private int eta;

    // Costruttore di default (senza parametri)
    public Persona() {
        this.nome = "Sconosciuto";
        this.eta = 0;
    }

    // Costruttore con parametri
    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    // Costruttore di copia
    public Persona(Persona altra) {
        this.nome = altra.nome;
        this.eta = altra.eta;
    }
}

// Uso
Persona p1 = new Persona();                    // default
Persona p2 = new Persona("Mario", 25);         // con parametri
Persona p3 = new Persona(p2);                  // copia di p2
```

5.2 Overloading dei Costruttori

Overloading = più metodi con lo stesso nome ma parametri diversi.

```
public class Rettangolo {
    private double larghezza;
    private double altezza;

    // Costruttore con due parametri
    public Rettangolo(double larghezza, double altezza) {
        this.larghezza = larghezza;
        this.altezza = altezza;
    }

    // Costruttore per un quadrato (un parametro)
    public Rettangolo(double lato) {
        this(lato, lato);  // chiama l'altro costruttore
    }

    // Costruttore di default (quadrato unitario)
    public Rettangolo() {
        this(1.0);  // chiama il costruttore con un parametro
    }

    // this() DEVE essere la prima istruzione!
}
```

5.3 Inizializzatori di Istanza

```java
public class Esempio {
    private int valore;

    // Inizializzatore di istanza (eseguito prima del costruttore)
    {
        System.out.println("Inizializzatore di istanza");
        valore = 10;
    }

    // Inizializzazione diretta
    private String nome = "Default";

    public Esempio() {
        System.out.println("Costruttore");
    }
}
// Ordine di esecuzione: inizializzazione diretta → inizializzatore → costruttore
```

5.4 Inizializzatori Statici

Gli **inizializzatori statici** vengono eseguiti una sola volta, quando la classe viene caricata:

```java
public class Database {
    private static String connectionUrl;

    // Inizializzatore statico
    static {
        System.out.println("Caricamento configurazione DB...");
        connectionUrl = "jdbc:mysql://localhost:3306/mydb";
        // Puoi fare operazioni complesse qui
    }

    public static String getConnectionUrl() {
        return connectionUrl;
    }
}
```

5.5 Finalize (Deprecato)

```java
// Il metodo finalize() era chiamato prima della garbage collection
// È DEPRECATO da Java 9! Usare try-with-resources o cleaner
@Override
protected void finalize() throws Throwable {
    try {
        // pulizia risorse
    } finally {
        super.finalize();
    }
}
```

5.6 Try-with-Resources (Java 7+)

```java
// Chiude automaticamente le risorse alla fine
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
```

```
} catch (IOException e) {
    e.printStackTrace();
}
// br viene chiuso automaticamente, anche in caso di eccezione!
```

---

6. Controllo dell'Accesso

6.1 Modificatori di Accesso

I modificatori controllano la visibilità di classi, metodi e attributi:

Modificatore	Classe Stessa	Stesso Package	Sottoclasse	Ovunque
`public`	✓	✓	✓	✓
`protected`	✓	✓	✓	✗
(package-private)	✓	✓	✗	✗
`private`	✓	✗	✗	✗

```
package com.esempio;

public class EsempioAccesso {
    public int pubblico;        // accessibile ovunque
    protected int protetto;     // accessibile nel package e nelle sottoclassi
    int pacchetto;              // accessibile solo nel package (default)
    private int privato;        // accessibile solo nella classe stessa

    public void metodoPubblico() {
        // Posso accedere a TUTTI i campi dalla classe stessa
        this.privato = 10;      // ✓ OK
        this.protetto = 20;     // ✓ OK
    }
}
```

6.2 Encapsulation (Incapsulamento)

L'**incapsulamento** nasconde i dettagli interni e espone un'interfaccia controllata:

```
public class ContoBancario {
    // Campi PRIVATI - non accessibili dall'esterno
    private double saldo;
    private String numeroConto;

    public ContoBancario(String numeroConto, double saldoIniziale) {
        this.numeroConto = numeroConto;
        this.saldo = saldoIniziale;
    }

    // GETTER: legge il valore
    public double getSaldo() {
        return saldo;
    }

    // Non c'è setter per saldo! Si modifica solo tramite metodi controllati

    public void deposita(double importo) {
        if (importo > 0) {
```

```
            saldo += importo;
            System.out.println("Depositati €" + importo + ". Nuovo saldo: €" + saldo);
        } else {
            System.out.println("Errore: importo non valido!");
        }
    }

    public boolean preleva(double importo) {
        if (importo > 0 && importo <= saldo) {
            saldo -= importo;
            System.out.println("Prelevati €" + importo + ". Nuovo saldo: €" + saldo);
            return true;
        }
        System.out.println("Saldo insufficiente o importo non valido!");
        return false;
    }
}
```

6.3 Perché l'Incapsulamento è Importante

```
public class TestConto {
    public static void main(String[] args) {
        ContoBancario conto = new ContoBancario("IT12345", 1000.0);

        // conto.saldo = -999999;  // ERRORE DI COMPILAZIONE! saldo è private

        conto.deposita(500);       // Depositati €500. Nuovo saldo: €1500
        conto.preleva(200);        // Prelevati €200. Nuovo saldo: €1300

        System.out.println("Saldo attuale: €" + conto.getSaldo());

        // Tentativi non validi gestiti correttamente:
        conto.deposita(-100);      // Errore: importo non valido!
        conto.preleva(999999);     // Saldo insufficiente!
    }
}
```

Vantaggi dell'incapsulamento:

- **Validazione**: controlli sui dati in ingresso
- **Flessibilità**: puoi cambiare l'implementazione interna senza rompere il codice esterno
- **Sicurezza**: protegge i dati sensibili

---

7. Riuso di Classi

7.1 Composizione: "HAS-A" (Ha un)

La **composizione** crea oggetti complessi combinando oggetti più semplici. Rappresenta una relazione "ha un".

```
// Componenti
public class Motore {
    private int cavalli;

    public Motore(int cavalli) {
        this.cavalli = cavalli;
    }
```

```
    public void avvia() {
        System.out.println("Motore di " + cavalli + " CV avviato!");
    }
}

public class Freno {
    public void applica() {
        System.out.println("Frenata in corso...");
    }
}

// Classe composta
public class Auto {
    // Composizione: Auto "ha un" Motore e "ha dei" Freni
    private Motore motore;      // UN motore
    private Freno[] freni;      // QUATTRO freni
    private String marca;

    public Auto(String marca, int cavalli) {
        this.marca = marca;
        this.motore = new Motore(cavalli);  // crea il motore
        this.freni = new Freno[4];           // crea 4 freni
        for (int i = 0; i < 4; i++) {
            this.freni[i] = new Freno();
        }
    }

    public void guida() {
        motore.avvia();
        System.out.println(marca + " in marcia!");
    }

    public void frena() {
        for (Freno freno : freni) {
            freno.applica();
        }
    }
}

// Uso
Auto auto = new Auto("Fiat", 75);
auto.guida();   // Motore di 75 CV avviato! Fiat in marcia!
auto.frena();   // Frenata in corso... (x4)
```

7.2 Ereditarietà: "IS-A" (È un)

L'**ereditarietà** crea una classe specializzata a partire da una classe generale. Rappresenta una relazione "è un".

```
// Classe base (superclasse)
public class Animale {
    protected String nome;  // protected: visibile nelle sottoclassi
    protected int eta;

    public Animale(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    public void emettiSuono() {
        System.out.println("Qualche suono generico...");
    }
```

```
    public void mangia() {
        System.out.println(nome + " sta mangiando.");
    }
}

// Classe derivata (sottoclasse)
public class Cane extends Animale {  // "Cane IS-A Animale"
    private String razza;

    public Cane(String nome, int eta, String razza) {
        super(nome, eta);  // chiama il costruttore della superclasse
        this.razza = razza;
    }

    // Override: sovrascrive il metodo della superclasse
    @Override
    public void emettiSuono() {
        System.out.println(nome + " abbia: Bau bau!");
    }

    // Metodo specifico del Cane
    public void scodinzola() {
        System.out.println(nome + " scodinzola felice!");
    }
}

// Altra sottoclasse
public class Gatto extends Animale {
    private String colore;

    public Gatto(String nome, int eta, String colore) {
        super(nome, eta);
        this.colore = colore;
    }

    @Override
    public void emettiSuono() {
        System.out.println(nome + " fa: Miao!");
    }

    public void faiFusa() {
        System.out.println(nome + " sta facendo le fusa...");
    }
}

// Uso
public class TestAnimali {
    public static void main(String[] args) {
        Cane cane = new Cane("Fido", 3, "Labrador");
        Gatto gatto = new Gatto("Whiskers", 2, "Grigio");

        cane.emettiSuono();   // Fido abbia: Bau bau!
        cane.mangia();        // Fido sta mangiando. (ereditato)
        cane.scodinzola();    // Fido scodinzola felice!

        gatto.emettiSuono();  // Whiskers fa: Miao!
        gatto.faiFusa();      // Whiskers sta facendo le fusa...
    }
}
```

7.3 Composizione vs Ereditarietà

Aspetto	Composizione (HAS-A)	Ereditarietà (IS-A)
Relazione	"ha un"	"è un"
Flessibilità	Più flessibile	Più rigida
Accoppiamento	Debole	Forte
Riutilizzo	Delega i metodi	Eredita direttamente
Quando usarla	Quando c'è una relazione di utilizzo	Quando c'è una vera specializzazione

Regola d'oro: Preferisci la **composizione** all'ereditarietà quando possibile!

---

8. Polimorfismo

8.1 Cos'è il Polimorfismo

Il **polimorfismo** (dal greco "molte forme") permette a oggetti di classi diverse di rispondere allo stesso messaggio in modo diverso. In Java si realizza tramite:

- **Override** di metodi (polimorfismo dinamico/runtime)
- **Overloading** di metodi (polimorfismo statico/compile-time)

8.2 Polimorfismo Dinamico (Method Overriding)

```
// Polimorfismo con ereditarietà
public class Forma {
    public void disegna() {
        System.out.println("Disegno una forma generica");
    }

    public double calcolaArea() {
        return 0;
    }
}

public class Cerchio extends Forma {
    private double raggio;

    public Cerchio(double raggio) {
        this.raggio = raggio;
    }

    @Override
    public void disegna() {
        System.out.println("Disegno un cerchio di raggio " + raggio);
    }

    @Override
    public double calcolaArea() {
        return Math.PI * raggio * raggio;
    }
}

public class Rettangolo extends Forma {
    private double larghezza, altezza;
```

```
    public Rettangolo(double larghezza, double altezza) {
        this.larghezza = larghezza;
        this.altezza = altezza;
    }

    @Override
    public void disegna() {
        System.out.println("Disegno un rettangolo " + larghezza + "x" + altezza);
    }

    @Override
    public double calcolaArea() {
        return larghezza * altezza;
    }
}

// Uso polimorfico
public class TestForme {
    public static void main(String[] args) {
        // Riferimento di tipo Forma, oggetto di tipo Cerchio
        Forma f1 = new Cerchio(5.0);
        Forma f2 = new Rettangolo(4.0, 6.0);

        // Stesso metodo, comportamento diverso!
        f1.disegna();        // Disegno un cerchio di raggio 5.0
        f2.disegna();        // Disegno un rettangolo 4.0x6.0

        System.out.println("Area f1: " + f1.calcolaArea());  // 78.54...
        System.out.println("Area f2: " + f2.calcolaArea());  // 24.0

        // Array polimorfico
        Forma[] forme = {new Cerchio(3), new Rettangolo(2, 5), new Cerchio(7)};

        double areaTotale = 0;
        for (Forma f : forme) {
            f.disegna();  // chiama la versione corretta!
            areaTotale += f.calcolaArea();
        }
        System.out.println("Area totale: " + areaTotale);
    }
}
```

8.3 Binding Dinamico (Late Binding)

Java usa il **late binding**: il metodo da chiamare viene deciso a **runtime** in base al tipo effettivo dell'oggetto, non del riferimento.

```
Forma f;           // riferimento di tipo Forma
f = new Cerchio(5); // oggetto reale è un Cerchio
f.disegna();        // chiama Cerchio.disegna() - deciso a runtime!

f = new Rettangolo(3, 4); // ora f punta a un Rettangolo
f.disegna();        // chiama Rettangolo.disegna() - deciso a runtime!
```

8.4 Polimorfismo Statico (Method Overloading)

L'**overloading** definisce più metodi con lo stesso nome ma firma diversa (parametri diversi).

```
public class Calcolatrice {

    // Overloading: stesso nome, parametri diversi
```

```
    public int somma(int a, int b) {
        return a + b;
    }

    public double somma(double a, double b) {
        return a + b;
    }

    public int somma(int a, int b, int c) {
        return a + b + c;
    }

    public String somma(String a, String b) {
        return a + b;  // concatenazione
    }
}

// Uso
Calcolatrice calc = new Calcolatrice();
calc.somma(5, 3);           // chiama int,int → 8
calc.somma(5.5, 3.2);       // chiama double,double → 8.7
calc.somma(1, 2, 3);        // chiama int,int,int → 6
calc.somma("Hello, ", "World!"); // chiama String,String
```

8.5 Casting e instanceof

```
Forma f = new Cerchio(5.0);

// f è un riferimento Forma, ma l'oggetto è un Cerchio
// Per usare metodi specifici di Cerchio, serve il casting
if (f instanceof Cerchio) {
    Cerchio c = (Cerchio) f;  // downcasting esplicito
    System.out.println("Raggio: " + c.getRaggio());
}

// Java 14+: pattern matching per instanceof
if (f instanceof Cerchio c) {
    // c è automaticamente disponibile come Cerchio
    System.out.println("Raggio: " + c.getRaggio());
}
```

---

9. Interfacce

9.1 Cos'è un'Interfaccia

Un'**interfaccia** è un contratto che specifica quali metodi una classe deve implementare, senza fornire l'implementazione. È come una "classe puramente astratta".

```
// Definizione di un'interfaccia
public interface Veicolo {
    // Tutti i metodi sono implicitamente public e abstract
    void avvia();
    void ferma();
    double getVelocitaMassima();
}

// Un'altra interfaccia
public interface Navigabile {
```

```
    void ancoraggio();
    void salpa();
}

// Implementazione: una classe IMPLEMENTA un'interfaccia
public class Automobile implements Veicolo {
    private double velocitaMassima;

    public Automobile(double velocitaMassima) {
        this.velocitaMassima = velocitaMassima;
    }

    @Override
    public void avvia() {
        System.out.println("Auto: giro la chiave, motore acceso!");
    }

    @Override
    public void ferma() {
        System.out.println("Auto: premo il freno, mi fermo.");
    }

    @Override
    public double getVelocitaMassima() {
        return velocitaMassima;
    }
}

// Implementazione multipla di interfacce (a differenza dell'ereditarietà!)
public class Anfibio implements Veicolo, Navigabile {

    @Override
    public void avvia() {
        System.out.println("Anfibio: motore acceso!");
    }

    @Override
    public void ferma() {
        System.out.println("Anfibio: mi fermo.");
    }

    @Override
    public double getVelocitaMassima() {
        return 80;
    }

    @Override
    public void ancoraggio() {
        System.out.println("Anfibio: getto l'ancora.");
    }

    @Override
    public void salpa() {
        System.out.println("Anfibio: salpo!");
    }
}
```

9.2 Interfacce vs Classi Astratte

Caratteristica	Interfaccia	Classe Astratta
Parola chiave	`interface`	`abstract class`
Metodi	Solo dichiarazioni (prima di Java 8)	Può avere metodi completi e astratti
Implementazione multipla	Sì	No (single inheritance)
Costruttori	No	Sì
Campi	Solo `public static final`	Qualsiasi tipo
Quando usarla	Quando serve definire un contratto	Quando serve una classe base con codice condiviso

9.3 Metodi Default nelle Interfacce (Java 8+)

```
public interface Stampabile {
    void stampa();

    // Metodo default: ha un'implementazione!
    default void stampaAnteprima() {
        System.out.println("--- ANTEPRIMA ---");
        stampa();
        System.out.println("--- FINE ANTEPRIMA ---");
    }

    // Metodo statico nell'interfaccia
    static void info() {
        System.out.println("Interfaccia per oggetti stampabili");
    }
}
```

9.4 Esercizio Pratico: Sistema di Pagamenti

```
// Interfaccia base
public interface MetodoPagamento {
    boolean paga(double importo);
    double getSaldo();
}

// Implementazioni concrete
public class CartaCredito implements MetodoPagamento {
    private String numeroCarta;
    private double saldo;

    public CartaCredito(String numeroCarta, double saldo) {
        this.numeroCarta = numeroCarta;
        this.saldo = saldo;
    }

    @Override
    public boolean paga(double importo) {
        if (saldo >= importo) {
            saldo -= importo;
            System.out.println("Pagato €" + importo + " con carta " + numeroCarta);
            return true;
        }
        System.out.println("Pagamento rifiutato: saldo insufficiente");
        return false;
    }

    @Override
```

```
    public double getSaldo() {
        return saldo;
    }
}

public class PayPal implements MetodoPagamento {
    private String email;
    private double saldo;

    public PayPal(String email, double saldo) {
        this.email = email;
        this.saldo = saldo;
    }

    @Override
    public boolean paga(double importo) {
        if (saldo >= importo) {
            saldo -= importo;
            System.out.println("Pagato €" + importo + " via PayPal (" + email + ")");
            return true;
        }
        return false;
    }

    @Override
    public double getSaldo() {
        return saldo;
    }
}

// Polimorfismo in azione!
public class Cassa {
    public void processaPagamento(MetodoPagamento metodo, double importo) {
        System.out.println("Importo da pagare: €" + importo);
        boolean successo = metodo.paga(importo);
        if (successo) {
            System.out.println("Pagamento accettato!");
        } else {
            System.out.println("Pagamento fallito!");
        }
    }
}

// Uso
Cassa cassa = new Cassa();
MetodoPagamento carta = new CartaCredito("1234-5678", 500);
MetodoPagamento paypal = new PayPal("user@email.com", 200);

cassa.processaPagamento(carta, 150);   // funziona con CartaCredito
cassa.processaPagamento(paypal, 50);   // funziona con PayPal!
```

---

10. Classi Interne

10.1 Tipi di Classi Interne

Una **classe interna** è una classe definita all'interno di un'altra classe. Esistono diversi tipi:

Tipo	Descrizione
Membro	Definita al livello dei campi, con visibilità controllata
Locale	Definita all'interno di un metodo
Anonima	Senza nome, creata e istanziata al volo
Statica	Classe interna dichiarata `static`, non ha riferimento all'outer

10.2 Classe Interna Membro

```
public class Outer {
    private String messaggio = "Hello from Outer!";

    // Classe interna membro
    public class Inner {
        public void stampa() {
            // Può accedere ai membri privati dell'outer!
            System.out.println(messaggio);
        }
    }

    public static void main(String[] args) {
        // Creazione: serve prima un'istanza di Outer
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.stampa();  // Hello from Outer!
    }
}
```

10.3 Classe Interna Locale

```
public class ClasseConLocale {
    private int valore = 10;

    public void metodo() {
        final int locale = 5;  // deve essere final o effectively final

        // Classe definita dentro un metodo!
        class Locale {
            public void stampa() {
                System.out.println("valore = " + valore);
                System.out.println("locale = " + locale);
            }
        }

        Locale l = new Locale();
        l.stampa();
    }
}
```

10.4 Classe Anonima

Le **classi anonime** sono classi senza nome, create e istanziate in un'unica espressione:

```
// Invece di creare una classe separata...
public class Cane implements Animale {
    @Override
    public void emettiSuono() {
```

```
        System.out.println("Bau!");
    }
}

// ...posso usare una classe anonima
Animale cane = new Animale() {
    @Override
    public void emettiSuono() {
        System.out.println("Bau!");
    }

    // posso anche aggiungere metodi e campi
    private String nome = "Fido";
};

cane.emettiSuono();  // Bau!
```

Uso comune con interfacce funzionali e listener:

```
// Con ActionListener (interfaccia funzionale)
JButton bottone = new JButton("Cliccami");
bottone.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Bottone cliccato!");
    }
});

// Java 8+: lambda expression (sintassi più compatta)
bottone.addActionListener(e -> System.out.println("Cliccato!"));
```

10.5 Classe Interna Statica

```
public class Outer {
    private static String staticMsg = "Messaggio statico";
    private String istanzaMsg = "Messaggio di istanza";

    // Classe interna statica
    public static class StaticInner {
        public void stampa() {
            // Può accedere SOLO ai membri statici dell'outer
            System.out.println(staticMsg);
            // System.out.println(istanzaMsg);  // ERRORE!
        }
    }
}

// Uso: NON serve un'istanza di Outer!
Outer.StaticInner inner = new Outer.StaticInner();
inner.stampa();
```

---

11. Strutture Dati: Collections Framework

11.1 Gerarchia delle Collections

```
                    Iterable
                       |
```



11.2 List: Lista Ordinata

```java
import java.util.*;

public class EsempioList {
    public static void main(String[] args) {
        // ArrayList: array dinamico, accesso veloce per indice
        List<String> arrayList = new ArrayList<>();
        arrayList.add("Mario");
        arrayList.add("Luigi");
        arrayList.add("Peach");
        arrayList.add(1, "Toad");  // inserisce in posizione 1

        System.out.println(arrayList.get(0));    // Mario
        System.out.println(arrayList.size());     // 4

        // LinkedList: lista concatenata, inserimento/cancellazione veloce
        List<Integer> linkedList = new LinkedList<>();
        linkedList.add(10);
        linkedList.add(20);
        linkedList.add(30);
        linkedList.remove(Integer.valueOf(20));  // rimuove l'elemento 20

        // Iterazione
        for (String nome : arrayList) {
            System.out.println(nome);
        }

        // Con iterator
        Iterator<String> it = arrayList.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

11.3 Set: Insieme di Elementi Unici

```java
public class EsempioSet {
    public static void main(String[] args) {
        // HashSet: insieme non ordinato, accesso O(1)
        Set<String> hashSet = new HashSet<>();
        hashSet.add("Roma");
        hashSet.add("Milano");
```

```
        hashSet.add("Napoli");
        hashSet.add("Roma");  // duplicato, ignorato!

        System.out.println(hashSet);  // [Roma, Milano, Napoli] (ordine non garantito)
        System.out.println(hashSet.contains("Roma"));  // true

        // TreeSet: insieme ordinato (ordinamento naturale)
        Set<Integer> treeSet = new TreeSet<>();
        treeSet.add(30);
        treeSet.add(10);
        treeSet.add(20);

        for (Integer n : treeSet) {
            System.out.println(n);  // 10, 20, 30 (ordinato!)
        }

        // LinkedHashSet: mantiene l'ordine di inserimento
        Set<String> linkedSet = new LinkedHashSet<>();
        linkedSet.add("Primo");
        linkedSet.add("Secondo");
        linkedSet.add("Terzo");
        System.out.println(linkedSet);  // [Primo, Secondo, Terzo]
    }
}
```

11.4 Map: Mappatura Chiave-Valore

```
public class EsempioMap {
    public static void main(String[] args) {
        // HashMap: mappa non ordinata
        Map<String, Integer> etaPersone = new HashMap<>();
        etaPersone.put("Mario", 30);
        etaPersone.put("Luigi", 25);
        etaPersone.put("Peach", 28);

        System.out.println(etaPersone.get("Mario"));  // 30
        System.out.println(etaPersone.containsKey("Bowser"));  // false

        // Iterazione sulle chiavi
        for (String nome : etaPersone.keySet()) {
            System.out.println(nome + " ha " + etaPersone.get(nome) + " anni");
        }

        // Iterazione sulle entry (più efficiente)
        for (Map.Entry<String, Integer> entry : etaPersone.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }

        // TreeMap: mappa ordinata per chiave
        Map<String, String> capitali = new TreeMap<>();
        capitali.put("Italia", "Roma");
        capitali.put("Francia", "Parigi");
        capitali.put("Germania", "Berlino");

        for (String paese : capitali.keySet()) {
            System.out.println(paese + " → " + capitali.get(paese));
        }
    }
}
```

11.5 Confronto tra Collection

Collection	Ordinata	Duplicati	Accesso	Uso Tipico
`ArrayList`	Sì (inserimento)	Sì	Casuale O(1)	Lista generica
`LinkedList`	Sì (inserimento)	Sì	Sequenziale	Code, stack
`HashSet`	No	No	O(1)	Insieme univoco
`TreeSet`	Sì (naturale)	No	O(log n)	Insieme ordinato
`HashMap`	No	Chiavi no	O(1)	Mappa generica
`TreeMap`	Sì (per chiave)	Chiavi no	O(log n)	Mappa ordinata

---

12. Gestione degli Errori: Eccezioni

12.1 Gerarchia delle Eccezioni

```
                Throwable
               /         \
          Error       Exception
                     /         \
          RuntimeException    Checked Exception
          (Unchecked)         (IOException, SQLException, ...)
```

Tipo	Descrizione	Esempio
Checked	Devono essere dichiarate o catturate	`IOException`, `SQLException`
Unchecked	Non devono essere dichiarate	`NullPointerException`, `ArrayIndexOutOfBoundsException`
Error	Problemi gravi del JVM, non catturabili	`OutOfMemoryError`, `StackOverflowError`

12.2 Try-Catch-Finally

```
public class GestioneEccezioni {

    public static void main(String[] args) {
        try {
            // Codice che potrebbe generare un'eccezione
            int risultato = 10 / 0;  // ArithmeticException!

        } catch (ArithmeticException e) {
            // Gestione specifica
            System.out.println("Errore: divisione per zero!");
            System.out.println("Messaggio: " + e.getMessage());

        } catch (Exception e) {
            // Gestione generica (cattura TUTTE le eccezioni)
            System.out.println("Errore generico: " + e);

        } finally {
            // Eseguito SEMPRE, con o senza eccezione
            System.out.println("Blocco finally eseguito");
        }
    }
}
```

12.3 Multiple Catch e Multi-Catch (Java 7+)

```
try {
    // codice rischioso
} catch (FileNotFoundException e) {
    System.out.println("File non trovato!");
} catch (IOException e) {
    System.out.println("Errore di I/O!");
}

// Java 7+: multi-catch
 try {
    // codice rischioso
} catch (FileNotFoundException | SQLException e) {
    // gestione comune per entrambe
    System.out.println("Errore: " + e.getMessage());
}
```

12.4 Throw e Throws

```
public class Calcolatrice {

    // throws: dichiara che il metodo PUÓ lanciare un'eccezione
    public double dividi(double a, double b) throws ArithmeticException {
        if (b == 0) {
            // throw: lancia esplicitamente un'eccezione
            throw new ArithmeticException("Divisione per zero non consentita!");
        }
        return a / b;
    }

    // Gestione con try-catch
    public double dividiSicuro(double a, double b) {
        try {
            return dividi(a, b);
        } catch (ArithmeticException e) {
            System.out.println("Errore: " + e.getMessage());
            return Double.NaN;
        }
    }
}
```

12.5 Eccezioni Personalizzate

```
// Crea la tua eccezione estendendo Exception (checked) o RuntimeException (unchecked)
public class SaldoInsufficienteException extends Exception {
    private double saldo;
    private double importoRichiesto;

    public SaldoInsufficienteException(double saldo, double importo) {
        super("Saldo insufficiente: saldo=" + saldo + ", richiesto=" + importo);
        this.saldo = saldo;
        this.importoRichiesto = importo;
    }

    public double getSaldo() { return saldo; }
    public double getImportoRichiesto() { return importoRichiesto; }
}
```

```java
// Uso
public class Conto {
    private double saldo;

    public void preleva(double importo) throws SaldoInsufficienteException {
        if (importo > saldo) {
            throw new SaldoInsufficienteException(saldo, importo);
        }
        saldo -= importo;
    }
}
```

12.6 Best Practices

```java
public class BestPractices {

    // 1. Cattura eccezioni specifiche, non generiche
    public void metodo1() {
        try {
            // codice
        } catch (FileNotFoundException e) {  // specifica
            // gestisci
        } catch (IOException e) {  // più generale
            // gestisci
        }
    }

    // 2. Non catturare Exception generica se non necessario
    // 3. Non lasciare catch vuoti!

    // 4. Usa try-with-resources per le risorse
    public void leggiFile(String path) {
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.err.println("Errore lettura file: " + e.getMessage());
        }
    }

    // 5. Documenta con @throws
    /**
     * Divide due numeri.
     * @param a dividendo
     * @param b divisore
     * @return risultato
     * @throws IllegalArgumentException se b è zero
     */
    public double dividi(double a, double b) {
        if (b == 0) throw new IllegalArgumentException("b non può essere 0");
        return a / b;
    }
}
```

---

13. Meta-Informazioni: Reflection

13.1 Cos'è la Reflection

La **reflection** (introspezione) permette a un programma Java di esaminare e manipolare la struttura del codice a **runtime**: classi, metodi, campi, costruttori.

13.2 Ottenere un Oggetto Class

```
// Tre modi per ottenere un oggetto Class

// 1. Metodo getClass() (da un'istanza)
String s = "Hello";
Class<?> classe1 = s.getClass();

// 2. Attributo .class (dal tipo)
Class<?> classe2 = String.class;

// 3. Class.forName() (dal nome completo)
Class<?> classe3 = Class.forName("java.lang.String");
```

13.3 Ispezione di Classe

```
import java.lang.reflect.*;

public class Ispezione {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = Class.forName("java.util.ArrayList");

        // Nome della classe
        System.out.println("Nome: " + clazz.getName());
        System.out.println("Semplice: " + clazz.getSimpleName());

        // Modificatori
        int modifiers = clazz.getModifiers();
        System.out.println("È public? " + Modifier.isPublic(modifiers));

        // Superclasse
        System.out.println("Superclasse: " + clazz.getSuperclass().getName());

        // Interfacce implementate
        Class<?>[] interfaces = clazz.getInterfaces();
        for (Class<?> i : interfaces) {
            System.out.println("Interfaccia: " + i.getName());
        }

        // Metodi pubblici
        Method[] methods = clazz.getMethods();
        for (Method m : methods) {
            System.out.println("Metodo: " + m.getName());
        }
    }
}
```

13.4 Invocazione Dinamica di Metodi

```
public class ReflectionEsempio {
    public static void main(String[] args) throws Exception {
        // Carica la classe dinamicamente
        Class<?> clazz = Class.forName("java.util.HashMap");
```

```
        // Crea un'istanza (costruttore senza parametri)
        Object instance = clazz.getDeclaredConstructor().newInstance();

        // Ottieni il metodo 'put'
        Method putMethod = clazz.getMethod("put", Object.class, Object.class);

        // Invoca il metodo sull'istanza
        putMethod.invoke(instance, "chiave", "valore");

        // Ottieni il metodo 'get'
        Method getMethod = clazz.getMethod("get", Object.class);
        Object result = getMethod.invoke(instance, "chiave");

        System.out.println(result);  // "valore"
    }
}
```

13.5 Accesso ai Campi Privati

```
public class AccessoPrivato {
    public static void main(String[] args) throws Exception {
        // Classe con campo privato
        class Persona {
            private String segreto = "Top Secret";
        }

        Persona p = new Persona();

        // Ottieni il campo privato
        Field field = Persona.class.getDeclaredField("segreto");

        // Rendi accessibile (bypassa il controllo di accesso!)
        field.setAccessible(true);

        // Leggi il valore
        String valore = (String) field.get(p);
        System.out.println("Valore privato: " + valore);  // Top Secret

        // Modifica il valore
        field.set(p, "Segreto rivelato!");
        System.out.println("Nuovo valore: " + field.get(p));
    }
}
```

13.6 Uso Pratico: Factory Dinamica

```
public class ReflectionFactory {

    // Crea un'istanza di una classe il cui nome è noto solo a runtime
    public static <T> T createInstance(String className) {
        try {
            Class<?> clazz = Class.forName(className);
            return (T) clazz.getDeclaredConstructor().newInstance();
        } catch (Exception e) {
            throw new RuntimeException("Impossibile creare istanza di " + className, e);
        }
    }

    public static void main(String[] args) {
        // Crea un ArrayList senza importarlo!
```

```
        List<String> list = createInstance("java.util.ArrayList");
        list.add("Hello Reflection!");
        System.out.println(list);
    }
}
```

---

14. Generics

14.1 Perché i Generics

I **generics** permettono di scrivere codice riutilizzabile con **type safety** a compile-time, evitando il casting esplicito.

```
// SENZA generics (codice vecchio pre-Java 5)
List listaVecchia = new ArrayList();
listaVecchia.add("Hello");
listaVecchia.add(123);       // OK a compile-time, pericoloso!
String s = (String) listaVecchia.get(0);  // cast necessario!

// CON generics (type-safe)
List<String> listaNuova = new ArrayList<>();
listaNuova.add("Hello");
// listaNuova.add(123);  // ERRORE DI COMPILAZIONE!
String s2 = listaNuova.get(0);  // nessun cast necessario!
```

14.2 Classe Generica

```
// T = type parameter (parametro di tipo)
public class Box<T> {
    private T contenuto;

    public void set(T contenuto) {
        this.contenuto = contenuto;
    }

    public T get() {
        return contenuto;
    }

    public static void main(String[] args) {
        // Box di Integer
        Box<Integer> boxInt = new Box<>();
        boxInt.set(42);
        int valore = boxInt.get();  // nessun cast!

        // Box di String
        Box<String> boxStr = new Box<>();
        boxStr.set("Hello");
        String testo = boxStr.get();

        // Il compilato controlla i tipi!
        // boxInt.set("testo");  // ERRORE DI COMPILAZIONE
    }
}
```

14.3 Metodo Generico

```java
public class Utilita {

    // Metodo generico: <T> prima del tipo di ritorno
    public static <T> void stampaArray(T[] array) {
        for (T elemento : array) {
            System.out.println(elemento);
        }
    }

    // Metodo generico con vincolo: T deve essere Number o sottoclasse
    public static <T extends Number> double somma(List<T> numeri) {
        double totale = 0;
        for (T n : numeri) {
            totale += n.doubleValue();
        }
        return totale;
    }

    public static void main(String[] args) {
        String[] nomi = {"Alice", "Bob"};
        Integer[] numeri = {1, 2, 3};

        stampaArray(nomi);    // T = String
        stampaArray(numeri);  // T = Integer

        // somma(List<String>) non compila!
        double risultato = somma(Arrays.asList(1, 2, 3, 4));
        System.out.println(risultato);  // 10.0
    }
}
```

14.4 Bounded Type Parameters

```java
// <T extends Number> : T deve essere Number o una sua sottoclasse
public class NumeroBox<T extends Number> {
    private T numero;

    public NumeroBox(T numero) {
        this.numero = numero;
    }

    public double getValore() {
        return numero.doubleValue();  // puoi chiamare metodi di Number!
    }
}

// Uso
NumeroBox<Integer> intBox = new NumeroBox<>(42);     // OK
NumeroBox<Double> doubleBox = new NumeroBox<>(3.14); // OK
// NumeroBox<String> strBox = ...  // ERRORE! String non estende Number
```

14.5 Wildcards

```java
// ? = unknown type (qualsiasi tipo)
public static void stampaListaWildcard(List<?> lista) {
    for (Object o : lista) {
        System.out.println(o);
    }
}
```

```java
// ? extends T = upper bounded (T o sottoclassi) - covarianza
public static double sommaWildcard(List<? extends Number> lista) {
    double totale = 0;
    for (Number n : lista) {
        totale += n.doubleValue();
    }
    return totale;
}

// ? super T = lower bounded (T o superclasses) - controvarianza
public static void aggiungiNumeri(List<? super Integer> lista) {
    lista.add(1);
    lista.add(2);
    lista.add(3);
}

// PECS: Producer Extends, Consumer Super
// - Leggere dalla lista → ? extends (producer)
// - Scrivere nella lista → ? super (consumer)
```

14.6 Generics vs Template C++

Aspetto	Java Generics	C++ Templates
Implementazione	Type erasure (tipi rimossi a runtime)	Code generation (codice generato per ogni tipo)
Primitivi	Non supportati (usare wrapper)	Supportati direttamente
Informazioni tipo a runtime	No (type erasure)	Sì
Specializzazione	No	Sì (template specialization)

---

15. Input/Output (I/O)

15.1 Stream in Java

Uno **stream** è un flusso sequenziale di dati. In Java ci sono due categorie:

Tipo	Classe Base	Uso
Byte Stream	`InputStream`, `OutputStream`	Dati binari (immagini, audio, file eseguibili)
Character Stream	`Reader`, `Writer`	Testo (file .txt, .csv, .xml)

15.2 Lettura e Scrittura di File di Testo

```java
import java.io.*;

public class FileTesto {

    // Scrivere su file (BufferedWriter)
    public static void scriviFile(String nomeFile, String contenuto) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(nomeFile))) {
            writer.write(contenuto);
            writer.newLine();
            writer.write("Seconda riga");
        } catch (IOException e) {
            System.err.println("Errore scrittura: " + e.getMessage());
```

```
        }
    }

    // Leggere da file (BufferedReader) - metodo più comune
    public static void leggiFile(String nomeFile) {
        try (BufferedReader reader = new BufferedReader(new FileReader(nomeFile))) {
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.err.println("Errore lettura: " + e.getMessage());
        }
    }

    // Leggere tutto il file in una stringa (Java 11+)
    public static String leggiTutto(String nomeFile) throws IOException {
        return Files.readString(Path.of(nomeFile));
    }
}
```

15.3 Stream di Byte per File Binari

```
public class FileBinario {

    // Copia di un file binario (immagine, video, ecc.)
    public static void copiaFile(String sorgente, String destinazione) {
        try (FileInputStream in = new FileInputStream(sorgente);
             FileOutputStream out = new FileOutputStream(destinazione)) {

            byte[] buffer = new byte[4096];  // buffer da 4KB
            int bytesLetti;

            while ((bytesLetti = in.read(buffer)) != -1) {
                out.write(buffer, 0, bytesLetti);
            }

            System.out.println("File copiato con successo!");

        } catch (IOException e) {
            System.err.println("Errore copia: " + e.getMessage());
        }
    }
}
```

15.4 Serializzazione

La **serializzazione** converte un oggetto in una sequenza di byte per salvarlo o trasmetterlo:

```
import java.io.*;

// La classe DEVE implementare Serializable
public class Persona implements Serializable {
    private static final long serialVersionUID = 1L;  // versione per compatibilità
    private String nome;
    private int eta;

    // transient: campo NON serializzato
    private transient String password;
```

```
    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}

public class Serializzazione {

    // Salva un oggetto su file
    public static void salvaOggetto(Object obj, String file) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file))) {
            out.writeObject(obj);
        }
    }

    // Carica un oggetto da file
    public static Object caricaOggetto(String file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(file))) {
            return in.readObject();
        }
    }
}
```

---

16. Tipi Enumerati

16.1 Cos'è un Enum

Un **enum** è un tipo speciale di classe che rappresenta un insieme fisso di costanti.

```
// Definizione semplice
public enum Giorno {
    LUNEDI, MARTEDI, MERCOLEDI, GIOVEDI,
    VENERDI, SABATO, DOMENICA
}

// Enum con campi e metodi
public enum StatoOrdine {
    NUOVO("In attesa"),
    PROCESSATO("In elaborazione"),
    SPEDITO("In consegna"),
    CONSEGNATO("Consegnato"),
    ANNULLATO("Annullato");

    private final String descrizione;

    // Costruttore privato (implicitamente)
    StatoOrdine(String descrizione) {
        this.descrizione = descrizione;
    }

    public String getDescrizione() {
        return descrizione;
    }
}

// Uso
public class TestEnum {
    public static void main(String[] args) {
        StatoOrdine stato = StatoOrdine.SPEDITO;
```

```
        System.out.println(stato.getDescrizione());  // In consegna

        // Confronto (puoi usare == per gli enum)
        if (stato == StatoOrdine.SPEDITO) {
            System.out.println("Il pacco è in viaggio!");
        }

        // Switch con enum
        switch (stato) {
            case NUOVO -> System.out.println("Appena ricevuto");
            case SPEDITO -> System.out.println("In transito");
            case CONSEGNATO -> System.out.println("Arrivato!");
            default -> System.out.println("Altro stato");
        }

        // Iterazione su tutti i valori
        for (StatoOrdine s : StatoOrdine.values()) {
            System.out.println(s + ": " + s.getDescrizione());
        }
    }
}
```

16.2 Metodi degli Enum

```
Giorno g = Giorno.LUNEDI;

System.out.println(g.name());           // "LUNEDI"
System.out.println(g.ordinal());        // 0 (posizione nell'enum)

// Conversione da String
Giorno converted = Giorno.valueOf("LUNEDI");  // Giorno.LUNEDI

// Tutti i valori
Giorno[] tutti = Giorno.values();  // array con tutti i valori
```

---

17. Annotazioni (Meta-programmazione)

17.1 Cos'è un'Annotazione

Le **annotazioni** sono metadati che forniscono informazioni aggiuntive al compilatore o a strumenti di sviluppo. Non influenzano direttamente l'esecuzione, ma possono essere lette a compile-time o runtime.

17.2 Annotazioni Predefinite

```
@Override       // Indica che stai sovrascrivendo un metodo
@Deprecated     // Indica che il metodo/classe è deprecato
@SuppressWarnings("unchecked")  // Sopprime warning del compilatore
@FunctionalInterface  // Indica che l'interfaccia ha un solo metodo astratto

// Esempi
public class AnnotazioniPredefinite {

    @Override
    public String toString() {
        return "Sovrascrivo toString() di Object";
    }

    @Deprecated
```

```
    public void vecchioMetodo() {
        // Usa nuovoMetodo() invece
    }

    @SuppressWarnings("unchecked")
    public void usaRawType() {
        List lista = new ArrayList();  // raw type, normalmente warning
        lista.add("test");
    }
}
```

17.3 Creare Annotazioni Personalizzate

```
import java.lang.annotation.*;

// Definizione dell'annotazione
@Retention(RetentionPolicy.RUNTIME)  // disponibile a runtime
@Target(ElementType.METHOD)          // può essere usata sui metodi
public @interface Test {
    // Parametri dell'annotazione (con valore di default)
    String descrizione() default "";
    int priorita() default 0;
}

// Annotazione per le classi
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Autore {
    String nome();
    String data();
}

// Uso
@Autore(nome = "Mario Rossi", data = "2024-01-15")
public class MiaClasse {

    @Test(descrizione = "Test del metodo principale", priorita = 1)
    public void metodoTest() {
        // codice del test
    }
}
```

17.4 Elaborazione delle Annotazioni con Reflection

```
public class AnnotationProcessor {

    public static void analizzaClasse(Class<?> clazz) {
        // Leggi annotazioni sulla classe
        if (clazz.isAnnotationPresent(Autore.class)) {
            Autore autore = clazz.getAnnotation(Autore.class);
            System.out.println("Autore: " + autore.nome());
            System.out.println("Data: " + autore.data());
        }

        // Leggi annotazioni sui metodi
        for (Method method : clazz.getDeclaredMethods()) {
            if (method.isAnnotationPresent(Test.class)) {
                Test test = method.getAnnotation(Test.class);
                System.out.println("Metodo test: " + method.getName());
                System.out.println("  Descrizione: " + test.descrizione());
```

```
                System.out.println("  Priorità: " + test.priorita());
            }
        }
    }
}
```

---

18. Programmazione Concorrente

18.1 Thread e Runnable

Un **thread** è un flusso di esecuzione indipendente all'interno di un programma.

```
// Metodo 1: estendere Thread
public class MioThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(getName() + ": " + i);
            try {
                Thread.sleep(1000);  // pausa di 1 secondo
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

// Metodo 2: implementare Runnable (preferito!)
public class MioRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(Thread.currentThread().getName() + ": " + i);
        }
    }
}

// Uso
public class TestThread {
    public static void main(String[] args) {
        // Con Thread
        MioThread t1 = new MioThread();
        t1.start();  // avvia il thread! (NON chiamare run() direttamente)

        // Con Runnable (più flessibile, puoi estendere un'altra classe)
        Thread t2 = new Thread(new MioRunnable());
        t2.start();

        // Con lambda (Java 8+)
        Thread t3 = new Thread(() -> {
            System.out.println("Thread con lambda!");
        });
        t3.start();
    }
}
```

18.2 Sincronizzazione

Quando più thread accedono a risorse condivise, serve la **sincronizzazione**:

```
public class Contatore {
    private int valore = 0;

    // Metodo sincronizzato: solo un thread alla volta può eseguirlo
    public synchronized void incrementa() {
        valore++;
    }

    // Blocco sincronizzato su un oggetto specifico
    public void decrementa() {
        synchronized (this) {
            valore--;
        }
    }

    public int getValore() {
        return valore;
    }
}

// Test con thread concorrenti
public class TestContatore {
    public static void main(String[] args) throws InterruptedException {
        Contatore contatore = new Contatore();

        // Creiamo 1000 thread che incrementano
        Thread[] threads = new Thread[1000];
        for (int i = 0; i < 1000; i++) {
            threads[i] = new Thread(contatore::incrementa);
            threads[i].start();
        }

        // Aspettiamo che finiscano tutti
        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Valore finale: " + contatore.getValore());
        // Senza synchronized potrebbe essere < 1000!
    }
}
```

18.3 Esempio Pratico: Simulazione Cassa Supermercato

```
import java.util.concurrent.*;

public class Supermercato {

    static class Cliente implements Runnable {
        private final int id;

        public Cliente(int id) {
            this.id = id;
        }

        @Override
        public void run() {
            System.out.println("Cliente " + id + " inizia a pagare");
```

```
            try {
                Thread.sleep((int)(Math.random() * 3000));  // tempo di pagamento
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("Cliente " + id + " ha finito!");
        }
    }

    public static void main(String[] args) {
        int numCasse = 3;
        int numClienti = 10;

        // Thread pool con 3 casse
        ExecutorService casse = Executors.newFixedThreadPool(numCasse);

        for (int i = 1; i <= numClienti; i++) {
            casse.execute(new Cliente(i));
        }

        casse.shutdown();
        try {
            casse.awaitTermination(1, TimeUnit.MINUTES);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Tutti i clienti serviti!");
    }
}
```

---

19. Maven

19.1 Cos'è Maven

Maven è uno strumento di gestione progetto e build automation per Java. Gestisce:

- Compilazione del codice
- Dipendenze (librerie esterne)
- Testing
- Packaging (JAR, WAR)
- Documentazione

19.2 Struttura di un Progetto Maven

```
mio-progetto/
├── pom.xml              <- file di configurazione principale
├── src/
│   ├── main/
│   │   ├── java/        <- codice sorgente
│   │   │   └── com/
│   │   │       └── esempio/
│   │   │           └── App.java
│   │   └── resources/   <- file di configurazione
│   └── test/
│       ├── java/        <- test unitari
```

```
│       └── resources/   <- risorse per i test
└── target/              <- output della compilazione (generato)
```

19.3 Il file POM.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Coordinate del progetto -->
    <groupId>com.esempio</groupId>
    <artifactId>mio-progetto</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>Mio Progetto</name>
    <description>Un esempio di progetto Maven</description>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <!-- Dipendenze -->
    <dependencies>
        <!-- JUnit per i test -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>

        <!-- Libreria per logging -->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.9</version>
        </dependency>
    </dependencies>

    <!-- Plugin di build -->
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

19.4 Comandi Maven Essenziali

```
# Creare un nuovo progetto
mvn archetype:generate -DgroupId=com.esempio -DartifactId=mio-progetto

# Compilare il progetto
mvn compile

# Eseguire i test
mvn test

# Creare il JAR
mvn package

# Installare nel repository locale
mvn install

# Pulire i file generati
mvn clean

# Combinare comandi
mvn clean package

# Saltare i test
mvn package -DskipTests
```

19.5 Gestione delle Dipendenze

Maven scarica automaticamente le librerie dal **Maven Central Repository**:

```
<dependencies>
    <!-- JDBC per MySQL -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>

    <!-- Libreria per JSON -->
    <dependency>
        <groupId>com.google.code.gson</groupId>
        <artifactId>gson</artifactId>
        <version>2.10.1</version>
    </dependency>
</dependencies>
```

Esercizi Pratici e Compiti d'Esame

Esercizio 1: Sistema di Gestione Studenti

```
// Interfaccia base
public interface Studente {
    String getMatricola();
    String getNome();
    double getMedia();
}
```

```java
// Classe astratta
public abstract class StudenteImpl implements Studente {
    protected String nome;
    protected String cognome;
    protected int annoImmatricolazione;

    // metodi comuni...
    protected String generaMatricola() {
        return "MAT" + annoImmatricolazione + "-" + System.currentTimeMillis();
    }
}

// Implementazioni concrete
public class StudenteTriennale extends StudenteImpl {
    private String corsoLaurea;

    public StudenteTriennale(String nome, String cognome, int anno, String corso) {
        this.nome = nome;
        this.cognome = cognome;
        this.annoImmatricolazione = anno;
        this.corsoLaurea = corso;
    }

    @Override
    public String getMatricola() {
        return "T" + generaMatricola();
    }

    @Override
    public double getMedia() {
        // calcolo specifico
        return 0.0;
    }
}

// Factory
public class StudenteFactory {
    public static Studente creaStudente(TipoStudente tipo, String nome, String cognome, int anno, String
extra) {
        return switch (tipo) {
            case TRIENNALE -> new StudenteTriennale(nome, cognome, anno, extra);
            case MAGISTRALE -> new StudenteMagistrale(nome, cognome, anno, extra);
        };
    }
}

// Enum
public enum TipoStudente {
    TRIENNALE, MAGISTRALE
}
```

Esercizio 2: Ereditarietà e Polimorfismo - Forme Geometriche

```java
public abstract class Forma {
    protected String colore;

    public Forma(String colore) {
        this.colore = colore;
    }

    public abstract double getArea();
```

```
    public abstract double getPerimetro();

    public String getColore() {
        return colore;
    }
}

public class Cerchio extends Forma {
    private double raggio;

    public Cerchio(String colore, double raggio) {
        super(colore);
        this.raggio = raggio;
    }

    @Override
    public double getArea() {
        return Math.PI * raggio * raggio;
    }

    @Override
    public double getPerimetro() {
        return 2 * Math.PI * raggio;
    }

    public double getRaggio() {
        return raggio;
    }
}

public class Rettangolo extends Forma {
    private double larghezza;
    private double altezza;

    public Rettangolo(String colore, double larghezza, double altezza) {
        super(colore);
        this.larghezza = larghezza;
        this.altezza = altezza;
    }

    @Override
    public double getArea() {
        return larghezza * altezza;
    }

    @Override
    public double getPerimetro() {
        return 2 * (larghezza + altezza);
    }
}

// Test
public class TestForme {
    public static void main(String[] args) {
        List<Forma> forme = new ArrayList<>();
        forme.add(new Cerchio("Rosso", 5));
        forme.add(new Rettangolo("Blu", 4, 6));
        forme.add(new Cerchio("Verde", 3));

        double areaTotale = 0;
        for (Forma f : forme) {
            System.out.println(f.getClass().getSimpleName() +
```

```
                            " (" + f.getColore() + "): " +
                            "Area=" + f.getArea());
            areaTotale += f.getArea();
        }
        System.out.println("Area totale: " + areaTotale);
    }
}
```

Esercizio 3: Gestione Eccezioni + I/O

```
public class GestioneStudentiFile {

    public static void salvaStudenti(List<Studente> studenti, String file) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(file))) {
            for (Studente s : studenti) {
                writer.println(s.getMatricola() + "," + s.getNome() + "," + s.getMedia());
            }
        } catch (IOException e) {
            System.err.println("Errore salvataggio: " + e.getMessage());
        }
    }

    public static List<Studente> caricaStudenti(String file) throws IOException {
        List<Studente> studenti = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String linea;
            while ((linea = reader.readLine()) != null) {
                String[] parti = linea.split(",");
                if (parti.length != 3) {
                    throw new FormatoFileException("Formato non valido: " + linea);
                }
                // crea studente...
            }
        }
        return studenti;
    }
}
```

Quick Reference - Sintassi Java Essenziale

Dichiarazioni

```
// Variabili
int x = 10;
double y = 3.14;
boolean flag = true;
char c = 'A';
String s = "Hello";

// Array
int[] numeri = {1, 2, 3, 4, 5};
String[] nomi = new String[10];

// ArrayList
List<String> lista = new ArrayList<>();
lista.add("elemento");
```

```java
// Map
Map<String, Integer> mappa = new HashMap<>();
mappa.put("chiave", 42);
```

Strutture di Controllo

```java
// If-else
if (condizione) {
    // ...
} else if (altraCondizione) {
    // ...
} else {
    // ...
}

// For
for (int i = 0; i < 10; i++) { }
for (Tipo elemento : collezione) { }

// While
while (condizione) { }
do { } while (condizione);

// Switch
switch (valore) {
    case 1 -> { }
    case 2 -> { }
    default -> { }
}
```

Classi e Oggetti

```java
public class NomeClasse extends SuperClasse implements Interfaccia1, Interfaccia2 {
    // campi
    // costruttori
    // metodi
}

// Creazione oggetto
Tipo oggetto = new Tipo(parametri);
```

Modificatori

Modificatore	Classe	Metodo	Campo
`public`	✓	✓	✓
`private`	(classe interna)	✓	✓
`protected`	✓	✓	✓
`static`	(classe interna)	✓	✓
`final`	✓	✓	✓
`abstract`	✓	✓	

> **Disclaimer:** Questi appunti sono stati creati per scopi didattici e rappresentano una sintesi del materiale del corso LMP (Primo Emicorso) di Tor Vergata. Per approfondimenti, consultare il libro "Thinking in Java" (4th Edition) di Bruce Eckel e i Java

> Tutorials ufficiali di Oracle.
